Abstract

Current token standards have provided the community with a platform on which to develop a decentralized economy that is focused on building Ethereum applications for the real world. As these applications mature and face consumer adoption, they begin to interface with corporate governance requirements as well as regulations. They must not only be able to meet corporate and regulatory requirements but must also be able to integrate with technology platforms underpinning their associated businesses. What follows is a simple and extendable standard that seeks to ease the burden of integration for wallets, exchanges, and issuers.

Motivation

Token issuers need a way to restrict transfers of tokens to be compliant with securities laws and other contractual obligations. This is most commonly required for ERC-20 tokens, but the same need applies to any token that moves a single amount between two accounts, such as ERC-777. Current implementations do not address these requirements.

A few examples:

- Enforcing Token Lock-Up Periods
- Enforcing Passed AML/KYC Checks
- Private Real-Estate Investment Trusts
- Delaware General Corporations Law Shares

Furthermore, standards adoption amongst token issuers has the potential to evolve into a dynamic and interoperable landscape of automated compliance.

The following design gives greater freedom / upgradability to token issuers and simultaneously decreases the burden of integration for developers and exchanges.

Additionally, this standard provides a pattern by which human-readable messages may be returned when token transfers are reverted. Transparency as to *why* a token's transfer was reverted is of equal importance to the successful enforcement of the transfer restriction itself.

Specification

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "NOT RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in RFC 2119 and RFC 8174.

ERC-1404 defines a transfer-restriction interface consisting of the two methods below. It is designed to be applied to a fungible token whose transfer moves a single `uint256` amount from one address to another — canonically ERC-20, and equally applicable to compatible token interfaces such as ERC-777.

A token claiming ERC-1404 compliance MUST implement the two methods below. It SHOULD preserve the function signatures and events of its underlying token interface (for example, ERC-20) and SHOULD NOT remove or alter them, so that integrators can continue to treat it as that interface; the restriction logic affects only whether a transfer succeeds, not the interface itself.

Methods

- **`detectTransferRestriction(address,address,uint256)`**

  Returns a restriction code for the proposed transfer of `value` tokens from `from` to `to`, or `0` if the transfer is unrestricted. The restriction logic is defined by the issuer.

  Enforcement MUST be consistent with `detectTransferRestriction` for the same state and inputs: a transfer MUST be rejected whenever `detectTransferRestriction` would return a non-zero code for that transfer, and MUST NOT be rejected for restriction reasons when it would return `0`.

  This requirement binds the contract that advertises `detectTransferRestriction` to callers and performs the transfer, whether it evaluates the policy itself or obtains the decision from one or more separate compliance contracts. Where the decision is obtained externally, that contract's `detectTransferRestriction` MUST report the aggregate outcome of every restriction source its transfer path consults, rather than the outcome of any single source; where its transfer path also rejects for restriction reasons of its own (for example a global transfer pause held by the token), those conditions MUST be reflected in its `detectTransferRestriction` as well. A contract that implements the restriction interface without performing transfers is addressed separately under Additional Specifications.

  - Implementations MAY satisfy this requirement by invoking `detectTransferRestriction` directly inside the token's transfer entry points (for ERC-20, `transfer` and `transferFrom`; for other token interfaces, the analogous transfer methods, such as ERC-777 `send`) and rejecting on a non-zero code. This is RECOMMENDED, because it guarantees the consistency above by construction.
  - Implementations MAY instead re-evaluate the equivalent conditions in the transfer path — including reverting with typed errors (for example, ERC-7943) rather than a restriction code — provided the observable outcome remains consistent with `detectTransferRestriction`.
  - When a transfer is rejected, reverting is RECOMMENDED; implementations MAY return `false` instead, consistently with the underlying token's transfer expectations.
  - A rejection that does not arise from the restriction policy — for example an insufficient balance, an insufficient allowance, or a recipient the underlying token interface itself forbids — is not a rejection "for restriction reasons" and does not require a non-zero code.

  `detectTransferRestriction` SHOULD NOT revert. An implementation that cannot evaluate its policy for the given inputs SHOULD return a non-zero restriction code denoting that condition, so that a caller can distinguish a restriction from an evaluation failure without interpreting a revert. Integrators SHOULD treat a reverting predictor as restricted.

  ```
  function detectTransferRestriction(address from, address to, uint256 value)
  external view returns (uint8);
  ```

- **`messageForTransferRestriction(uint8)`**

  Returns the human-readable message corresponding to `restrictionCode`.

  For the reserved code `0`, this SHOULD return a message denoting the absence of a restriction (for example, `"No restriction"`) and SHOULD NOT report `0` as an unknown or invalid code.

For a code the implementation does not recognize, this SHOULD return a deterministic, non-empty human-readable string indicating that the code is unrecognized, and SHOULD NOT revert, so that a caller enumerating codes to build a display table does not have to interpret a revert. That string SHOULD NOT denote the absence of a restriction: an unrecognized non-zero code still denotes a restriction, one whose meaning the implementation cannot resolve, and presenting it as unrestricted would misreport a blocked transfer as permitted.

An implementation SHOULD define a message for every code its restriction methods can return. Where a code originates from a separate compliance contract, the implementation that surfaces the code to callers is the one responsible for resolving it to a message; this specification does not prescribe the mechanism by which it does so.

```
function messageForTransferRestriction(uint8 restrictionCode) external view
returns (string memory);
```

Extension: spender-aware restriction detection (OPTIONAL)

`detectTransferRestriction(address,address,uint256)` describes a transfer solely in terms of `from`, `to`, and `value`. It has no `spender` parameter and therefore cannot express a restriction that depends on the party *initiating* a delegated transfer.

For ERC-20, `transferFrom` is executed by a `spender` that may differ from `from`; a policy that restricts the spender itself — for example a frozen operator, or an operator that has not passed its own AML/KYC check — is invisible to `detectTransferRestriction`.

An integrator that predicts a `transferFrom` outcome by calling only `detectTransferRestriction` will therefore mispredict exactly these cases.

An implementation MAY expose a spender-aware companion method:

```
function detectTransferRestrictionFrom(address spender, address from, address
to, uint256 value) external view returns (uint8);
```

`detectTransferRestrictionFrom` returns a restriction code for the delegated transfer of `value` tokens from `from` to `to` initiated by `spender`, or `0` if the transfer is unrestricted.

It shares the restriction code space of `detectTransferRestriction`, and its codes are looked up through the same `messageForTransferRestriction`.

An implementation that exposes `detectTransferRestrictionFrom`:

- MUST keep it consistent with delegated-transfer enforcement, in the same sense the base method is consistent with the transfer path: a `transferFrom(from, to, value)` executed by `spender` MUST be rejected whenever `detectTransferRestrictionFrom(spender, from, to, value)` would return a non-zero code, and MUST NOT be rejected for restriction reasons when it would return `0`.
- SHOULD evaluate the `spender == from` case through the spender-aware path, because `transferFrom` is a distinct delegated-transfer entry point whose authorization may depend on the initiating operator even when that operator equals `from`.
    - An implementation MAY instead evaluate `spender == from` through the non-spender-aware path — equivalently, skip the spender-specific checks — as an optimization, but

only when doing so is observably equivalent, i.e. its policy imposes no restriction on the initiating operator's identity beyond the `from` / `to` / `value` conditions.
    - An implementation whose policy *does* restrict the operator identity (for example, an allow-listed or frozen operator set) MUST NOT skip. When the policy does not distinguish operator identity from ownership, `detectTransferRestrictionFrom(from, from, to, value)` and `detectTransferRestriction(from, to, value)` return the same code, and skipping is always safe. The divergence that skipping introduces for operator-restricting policies is described in Security Considerations.
- MUST NOT change the meaning of, or the enforcement consistency required of, `detectTransferRestriction`. The base method continues to describe the `from` / `to` / `value` conditions; the extension adds only the `spender` dimension, so a caller that does not care about the spender can keep using the base method unchanged.

The extension is OPTIONAL and does not change the mandatory ERC-1404 interface identifier `0xab84a5c8`, which is the exclusive-or of the selectors of the two mandatory methods alone.

The extension is advertised under a *second, separate* identifier, `0x78a8de7d`, which is the exclusive-or of the selectors of **all three** methods — `detectTransferRestriction`, `messageForTransferRestriction`, and `detectTransferRestrictionFrom`.

An implementation that exposes `detectTransferRestrictionFrom` and supports ERC-165:

- MUST return `true` for `0xab84a5c8`, exactly as a non-extended implementation does — the mandatory interface is still fully present, so a base-only integrator continues to detect it.
- MUST return `true` for `0x78a8de7d`, so that an integrator can detect the spender-aware predictor by a single `supportsInterface` call rather than probing for it and interpreting a revert.

The extension identifier deliberately covers all three selectors rather than `detectTransferRestrictionFrom` alone.

- An identifier over the single added method would let a contract advertise the spender-aware predictor without asserting the mandatory pair it depends on; folding the two base selectors into the extension identifier keeps it self-contained, so `supportsInterface(0x78a8de7d) == true` is sufficient evidence that all three methods are present.
- Because it spans all three selectors, this identifier is their explicit exclusive-or, not that of the single added method alone; the Reference Implementation shows how it is computed and pins the value under test.

Additional Specifications

- Implementations MAY add ERC-165 interface detection support for ERC-1404.
- Implementations MAY apply analogous restriction checks to token supply-changing operations (for example, mint and burn). Such checks are OPTIONAL and are not required for ERC-1404 compliance. Where they are applied, the following hold.
    - **Encoding.** An implementation that evaluates a supply-changing operation through the restriction methods of this specification MUST encode a mint as `from == address(0)` and a burn as `to == address(0)`, matching the ERC-20 `Transfer` event convention for issuance and redemption. This makes the operation expressible in the three-parameter signature and lets an off-chain caller construct the same query the token

evaluates. An implementation MAY instead expose separate predictors for supply-changing operations, in which case this encoding does not apply to it.
    - **Operator.** Where the implementation also exposes the optional spender-aware extension, the `spender` parameter of `detectTransferRestrictionFrom` MAY carry the operator initiating the mint or burn, so that the policy can restrict that operator directly.
    - **Consistency.** The enforcement-consistency requirement of `detectTransferRestriction` applies to these operations as it does to transfers: the operation MUST be rejected whenever the predictor the implementation designates for it would return a non-zero code for the same state and inputs, and MUST NOT be rejected for restriction reasons when that predictor would return `0`.
    - **Predictor selection.** An implementation that exposes both predictors MUST document which one governs each supply-changing entry point, and enforcement MUST agree with the documented one. The two need not agree with each other: a mint entry point that carries no operator parameter is governed by `detectTransferRestriction(address(0), to, value)`, and predicting it with `detectTransferRestrictionFrom` may report a restriction that the entry point never enforces.
- The ERC-165 interface identifier for ERC-1404 is `0xab84a5c8` (the two mandatory methods). The identifier for the optional spender-aware extension is `0x78a8de7d` (all three methods, including `detectTransferRestrictionFrom`); an implementation exposing the extension advertises both, as described under Extension: spender-aware restriction detection.
- Compliance-related contracts (for example, a rule engine or compliance module) MAY implement the ERC-1404 restriction interface without implementing any token interface at all. Such a contract conforms to this specification as a **restriction source**: `detectTransferRestriction` and `messageForTransferRestriction` MUST behave as specified above, but the enforcement-consistency requirement does not apply to it, because it performs no transfers and therefore has no enforcement path of its own.
- A contract that performs transfers and advertises ERC-1404 is a **restricted token** and remains fully bound by the enforcement-consistency requirement, including where it obtains its restriction decisions from one or more restriction sources. Consulting a restriction source does not transfer that obligation to the source: the restricted token is the contract an integrator queries, so it is the contract whose reported code MUST match what its transfer path does.
- Restriction checks SHOULD be deterministic for the same state and inputs, so that the reporting and enforcement results remain consistent. Guidance on the data a restriction policy relies on is given in Security Considerations and is non-normative.
- The string returned by `messageForTransferRestriction` SHOULD NOT be treated as an authorization primitive.
- This specification reserves only code `0`, which denotes the absence of a restriction. It assigns no meaning to codes `1` through `255`; their interpretation is defined by the issuer's policy and is resolved through `messageForTransferRestriction`.
- Implementations SHOULD define and manage restriction code allocation carefully, because `uint8` limits the available code space to 256 values (`0` to `255`). The code space cannot be partitioned across independently authored policies, so where a restricted token aggregates several restriction sources, resolving collisions between the codes they return — and

mapping each surfaced code back to a message — is the aggregating implementation's responsibility.

Rationale

The standard proposes two functions on top of an underlying token interface (canonically ERC-20). The rationale for each is described below.

1. `detectTransferRestriction` - This function encodes the restriction logic of an issuer's token transfers. Some examples of this might include, checking if the token recipient is whitelisted, checking if a sender's tokens are frozen in a lock-up period, etc.
    - Because implementation is up to the issuer, this function serves to standardize the *result* that transfer enforcement must agree with, rather than mandating a specific call site: an implementation may invoke it directly inside the transfer path or re-evaluate the equivalent conditions, as long as the outcome is consistent (see Specification).
    - Additionally, 3rd parties may publicly call this function to check the expected outcome of a transfer.
    - Because this function returns a `uint8` code rather than a boolean or just reverting, it allows the function caller to know the reason why a transfer might fail and report this to relevant counterparties.
2. `messageForTransferRestriction` - This function is effectively an accessor for the "message", a human-readable explanation as to *why* a transaction is restricted. By standardizing message look-ups, we empower user interface builders to effectively report errors to users.
3. Optional ERC-165 support - Implementations may expose ERC-165 support for interface discovery.
4. Token-agnostic interface - Although ERC-1404 was originally designed as an ERC-20 extension, its two methods only assume a transfer that moves a single `uint256` amount between two addresses. The interface is therefore reused unchanged with other compatible token interfaces such as ERC-777, and can be implemented by a standalone compliance contract that is itself not a token. The standard deliberately does not require ERC-20, so that it can describe the restriction behavior independently of the token it is attached to.
5. Optional spender-aware detection - The mandatory `detectTransferRestriction` is kept at three parameters (`from`, `to`, `value`) so that it stays token-agnostic and matches the original ERC-1404 signature that integrators already call.
    - The `spender` dimension needed to predict a delegated transfer (`transferFrom`) is therefore split into an OPTIONAL extension rather than added to the base method, which would have broken its selector and its ERC-165 identifier and imposed a parameter on the many policies that do not restrict the spender. Splitting it also lets an integrator detect, via a distinct ERC-165 identifier, whether a given token can predict `transferFrom` outcomes at all — a token without the extension simply cannot, and the integrator learns this rather than silently mispredicting. The extension does not require `detectTransferRestrictionFrom(from, from, …)` to equal `detectTransferRestriction(from, …)`.
    - A `transferFrom` initiated by the holder is still a delegated transfer — it flows through the allowance path and is a different entry point than a direct `transfer` — so a policy that restricts the initiating operator (even when the operator is the holder) is legitimate, and the spender-aware predictor must be free to describe it. The only invariant that matters for integrators is that `detectTransferRestrictionFrom` agree with

`transferFrom` enforcement; requiring it to instead mirror the direct-transfer predictor would force it to misreport, and for operator-restricting policies would conflict with that agreement.
    - Implementations whose policy does not distinguish operator identity from ownership will observe the two predictors coincide for `spender == from` and MAY skip the spender-aware evaluation as an optimization.
6. Restriction sources and aggregation - The deployed shape of a permissioned token is frequently a token delegating to a separate, independently upgradeable rule engine, which may itself compose several policies. Defining a restriction source as a distinct conformance class keeps that architecture explicitly in scope without weakening the guarantee integrators rely on: the enforcement-consistency requirement stays attached to the contract that actually performs the transfer, because that is the contract an integrator queries and the only one whose report can be checked against an outcome. Requiring the restricted token to report the *aggregate* outcome, rather than the outcome of any single source, is what keeps the guarantee meaningful once a transfer path consults more than one source — a token that forwards one engine's `0` while its own pause flag blocks the transfer would otherwise be conformant while misreporting.
    - The corollary is that code allocation cannot be solved at the level of this specification. A `uint8` space is not partitionable across policies authored independently of one another, and any allocation convention this specification imposed would be unenforceable and would age badly. The requirement is therefore placed where it can be met and tested: the aggregating implementation resolves collisions within its own deployment and answers for every code it surfaces.
7. Mint and burn - Restriction checks on supply-changing operations remain OPTIONAL, because the original ERC-1404 did not define them and requiring them would strand conformant implementations. But an implementation that opts in and evaluates them through this interface needs a way to express "there is no sender" in a signature that has no room for one, and integrators need to construct the same query the token evaluates. The `address(0)` encoding is specified rather than invented: ERC-20 already establishes `address(0)` as the mint and burn counterparty in its `Transfer` event, so this records existing practice instead of adding a convention. Extending the enforcement-consistency requirement to these operations costs nothing for implementations that do not opt in, and closes the case where an implementation reports a mint restriction it does not enforce — the operation permissioned issuers most depend on.

Backwards Compatibility

By design ERC-1404 only adds new methods and leaves the underlying token interface untouched, so it is interface-compatible with ERC-20 (and other compatible token interfaces such as ERC-777). Transfer restrictions may, however, introduce behavioral differences, because otherwise-valid transfers can be rejected.

This revision is also compatible with the original ERC-1404 text: an implementation conformant with that text remains conformant here, and no deployed implementation is invalidated.

- The two mandatory method signatures are unchanged, so no deployed ERC-165 identifier, ABI, or integration breaks.
- The original required `detectTransferRestriction` to be evaluated inside `transfer` and `transferFrom` and the transaction to be reverted on a non-zero code. An implementation that does so satisfies the enforcement-consistency requirement by construction, and is the

aggregate of its own single restriction source, so the requirement on delegating implementations is vacuous for it.
- Every other requirement this revision adds is either stated at SHOULD level — the treatment of unrecognized codes, message coverage, and not reverting from `detectTransferRestriction` — or conditional on a feature the original did not define: ERC-165 signaling, restriction checks on supply-changing operations, and the spender-aware extension. An implementation that uses none of them is unaffected by all of them.
- The original left the meaning of codes other than `0` to the issuer, and this revision states that explicitly rather than narrowing it.

The one substantive obligation this revision adds for existing deployments is directed at implementations the original did not describe: a token that obtains its restriction decisions from a separate compliance contract and reports that contract's code verbatim, while its own transfer path can reject for further reasons, now misreports and must aggregate instead. Such an implementation was outside the original's scope rather than permitted by it.

Test Cases

The following table-driven cases SHOULD be verified for every implementation. The examples use a whitelist-based policy (code `0` = no restriction, `1` = sender not whitelisted, `2` = recipient not whitelisted) to keep the expected values concrete, but the same structure applies to any issuer-defined policy.

`detectTransferRestriction`

Scenario	Expected return
Both `from` and `to` satisfy the policy	`0`
`from` violates the policy	Non-zero restriction code
`to` violates the policy; `from` does not	Non-zero restriction code distinct from the sender case
Both `from` and `to` violate the policy	Non-zero code; the specific code returned depends on the implementation's evaluation order
`value` is `0`, and `from` and `to` satisfy the policy	Whatever the implementation's documented policy prescribes; the enforcement path MUST agree with it, so a zero-value transfer is rejected if and only if a non-zero code is returned
`from == to`, and both satisfy the policy	As above: the returned code and the enforcement outcome MUST agree
The implementation cannot evaluate its policy for the given inputs	A non-zero code denoting that condition, rather than a revert

`detectTransferRestrictionFrom` (optional extension)

For implementations that expose the spender-aware extension:

Scenario	Expected return
`spender`, `from`, and `to` all satisfy the policy	`0`
`spender == from`, policy does **not** restrict operator identity	Equal to `detectTransferRestriction(from, to, value)`
`spender == from`, policy **does** restrict the initiating operator	Reflects the delegated-transfer policy for that operator, and is consistent with `transferFrom` enforcement; MAY differ from `detectTransferRestriction(from, to, value)`
A spender-specific restriction applies (e.g., `spender` is frozen); `from` and `to` satisfy the policy	Non-zero code. Note `detectTransferRestriction(from, to, value)` MAY still return `0` for the same `from`/`to`/`value`, because it cannot observe the spender — this divergence is expected, not a defect
`from` or `to` violates the policy, and no spender-specific restriction applies	The same non-zero code `detectTransferRestriction` returns for that condition
Both a spender-specific restriction and a `from`/`to` violation apply	Non-zero; which of the two codes is returned depends on the implementation's evaluation order, as for the base method

`messageForTransferRestriction`

Input	Expected output
`0`	A deterministic human-readable string indicating no restriction (e.g., `"No restriction"`)
Any known restriction code	The corresponding deterministic human-readable message
Any code the implementation does not recognize	A deterministic, non-empty string marking the code as unrecognized; distinct from the code-`0` message, and not a revert
Every code the implementation's restriction methods can return	A message, in each case (no code is surfaced without one)

Transfer enforcement

Scenario	Expected behavior for `transfer` and `transferFrom`
`detectTransferRestriction` returns `0`	Transfer succeeds
`detectTransferRestriction` returns a non-zero code	Transfer reverts (preferred) or returns `false`

For implementations that expose the optional extension, `transferFrom` enforcement is additionally checked against the spender-aware predictor:

Scenario	Expected behavior for `transferFrom` executed by `spender`
`detectTransferRestrictionFrom(spender, from, to, value)` returns `0`	`transferFrom` succeeds
`detectTransferRestrictionFrom(spender, from, to, value)` returns a non-zero code	`transferFrom` reverts (preferred) or returns `false`

Supply-changing operations (optional)

For implementations that apply restriction checks to mint and burn through this interface:

Scenario	Expected behavior
`detectTransferRestriction(address(0), to, value)` returns `0`	The mint succeeds
`detectTransferRestriction(address(0), to, value)` returns a non-zero code	The mint is rejected
`detectTransferRestriction(from, address(0), value)` returns `0`	The burn succeeds
`detectTransferRestriction(from, address(0), value)` returns a non-zero code	The burn is rejected
`to` satisfies the recipient policy but is not otherwise permitted to send	The mint still succeeds — the mint query carries no sender, so a sender-side condition MUST NOT be applied to `address(0)` as though it were a holder

Where the implementation designates the spender-aware predictor for a supply-changing entry point, substitute `detectTransferRestrictionFrom(operator, address(0), to, value)` (mint) or `detectTransferRestrictionFrom(operator, from, address(0), value)` (burn) in the rows above.

ERC-165 interface detection (optional)

For implementations that expose ERC-165 support:

Input to `supportsInterface`	Expected return
`0xab84a5c8` (ERC-1404 interface identifier)	`true`
`0x01ffc9a7` (ERC-165 interface identifier)	`true`
Any unrecognized selector	`false`

For implementations that also expose the optional spender-aware extension:

Input to `supportsInterface`	Expected return
`0xab84a5c8` (mandatory ERC-1404 methods)	`true`
`0x78a8de7d` (extension: all three methods, including `detectTransferRestrictionFrom`)	`true`
`0x01ffc9a7` (ERC-165 interface identifier)	`true`
Any unrecognized selector	`false`

A complete Foundry test suite covering all the cases above is available in the assets folder, split across `test/ERC1404.t.sol`, `test/ERC1404SpenderAware.t.sol` and `test/WhitelistRuleEngine.t.sol`.

Reference Implementation

A complete reference implementation built with Foundry and OpenZeppelin Contracts v5 is provided in the assets folder:

File	Description
`src/IERC1404.sol`	Interface — extends `IERC20` with the two ERC-1404 functions
`src/ERC1404.sol`	Restricted token — whitelist-based, with ERC-165 support and mint/burn checks
`src/IERC1404SpenderAware.sol`	Interface for the optional spender-aware extension
`src/ERC1404SpenderAware.sol`	Restricted token exposing `detectTransferRestrictionFrom` and both interface identifiers
`src/engine/IERC1404Restriction.sol`	The two restriction methods without `IERC20` — the restriction-source interface

File	Description
src/engine/WhitelistRuleEngine.sol	Restriction source — a standalone rule engine that is not a token
src/engine/RestrictedToken.sol	Restricted token delegating to a rule engine, aggregating it with a policy of its own
test/ERC1404.t.sol	Foundry test suite covering all mandatory behaviors
test/ERC1404SpenderAware.t.sol	Tests for the optional spender-aware extension
test/WhitelistRuleEngine.t.sol	Tests for the restriction-source and aggregation behaviors

The concrete implementation applies a whitelist policy and defines the following restriction codes. Codes `1` and `2` are specific to this implementation, as ERC-1404 assigns no meaning to codes other than `0`.

Code	Constant	Message
`0`	`TRANSFER_OK`	`"No restriction"`
`1`	`SENDER_NOT_WHITELISTED`	`"Sender not whitelisted"`
`2`	`RECIPIENT_NOT_WHITELISTED`	`"Recipient not whitelisted"`

Notable design decisions in this implementation:

- `transfer` and `transferFrom` revert with a typed `TransferRestricted(uint8 code, string message)` error on non-zero codes, rather than returning `false`.
- `detectTransferRestriction` checks the sender before the recipient, so callers can distinguish the two failure cases with a single view call before submitting a transaction.
- `supportsInterface(0xab84a5c8)` returns `true`, enabling on-chain interface discovery.
- Mint and burn apply the same policy through `detectTransferRestriction`, using the `address(0)` encoding: a mint is queried as `detectTransferRestriction(address(0), to, value)` and a burn as `detectTransferRestriction(from, address(0), value)`. The policy branches on that encoding rather than requiring `address(0)` to be allow-listed, so issuance never depends on making the zero address an acceptable transfer counterparty.
- `messageForTransferRestriction` returns `"Unknown restriction code"` for any code it does not define, and does not revert.
- `RestrictedToken` shows the aggregation requirement: it holds a pause flag of its own and delegates the address policy to `WhitelistRuleEngine`, so its `detectTransferRestriction` reports the outcome of both sources rather than forwarding the engine's code unchanged, and its `messageForTransferRestriction` resolves its own code before delegating the rest to the engine.
- Implementations that also conform to ERC-7943 will likely revert with that standard's typed errors rather than `TransferRestricted(uint8 code, string message)`. Those errors do not carry a restriction code or human-readable message.

This example is provided for educational purposes only and has not been audited. Do not use in production without a thorough independent security review.

Security Considerations

- Implementations are expected to encode policy in `detectTransferRestriction`, so mistakes in this logic can block valid transfers or allow restricted transfers.
- Restriction checks that are not deterministic for the same state and inputs can create inconsistent behavior between the reporting and enforcement paths.
- Because `detectTransferRestriction` is a `view` function, any result obtained off-chain before a transfer is a prediction: the relevant state may change before the transfer executes, so a transfer predicted to succeed may later be rejected (and vice-versa). Implementations and integrators MUST treat the on-chain enforcement at execution time as authoritative, not an earlier off-chain reading.
- The following is informational guidance, not a conformance requirement. Because the policy in `detectTransferRestriction` is defined entirely by the issuer, its security depends on the data that policy reads. A policy that branches on values the transacting party can influence within the transfer transaction — for example a spot price read from an automated market maker, which a flash loan can move and restore in a single transaction — can be forced to evaluate as unrestricted at execution time, even though the check runs during the transfer. Issuers are encouraged to base restriction decisions on state they control (such as whitelist or frozen flags) rather than on inputs a counterparty can manipulate.
- When an implementation enforces transfers by re-evaluating the restriction conditions separately rather than calling `detectTransferRestriction` directly (for example, to revert with typed errors such as ERC-7943), the reporting function and the enforcement path become two codepaths that can drift apart. If they diverge, `detectTransferRestriction` may return `0` for a transfer that reverts, or a non-zero code for a transfer that succeeds, defeating the purpose of the machine-readable code for exchanges and user interfaces. Such implementations SHOULD treat the equivalence between `detectTransferRestriction` and the enforcement path as an invariant and verify it under test (for example, asserting over many inputs that a non-zero code holds if and only if the transfer is rejected).
- `detectTransferRestriction` is blind to the spender of a delegated transfer. A `transferFrom` can be rejected for a spender-specific reason (for example a frozen operator) that `detectTransferRestriction(from, to, value)` cannot see, so it may return `0` for a `transferFrom` that then reverts. This is not a violation of the enforcement-consistency requirement, which is defined over the `from`/`to`/`value` conditions the base method describes; it is a limitation of the three-parameter signature. Integrators MUST NOT treat a `0` from `detectTransferRestriction` as a prediction that a `transferFrom` will succeed. Where spender-aware prediction is required, use the optional `detectTransferRestrictionFrom` extension if the token exposes it; a token that does not expose it cannot predict `transferFrom` outcomes on-chain, and integrators MUST fall back to submitting the transfer and handling the revert.
- Because `transferFrom` is a delegated-transfer entry point, an implementation that restricts the initiating operator SHOULD evaluate `spender == from` through the spender-aware path rather than collapsing it to the direct-transfer predictor. Collapsing it (for example, an unconditional `if (spender == from) return detectTransferRestriction(from, to, value);` short-circuit) can make `detectTransferRestrictionFrom` return `0` for a self-initiated `transferFrom` that enforcement then rejects, reintroducing a

reporting/enforcement divergence. The non-spender-aware evaluation is a safe optimization *only* for policies that do not restrict operator identity beyond ownership.

- The correspondence between predictor and entry point runs in both directions, and mispredicting is possible either way. The base method under-reports for delegated transfers, as described above. Symmetrically, the spender-aware method over-reports for an entry point that carries no initiating operator distinct from `from` — for example a mint entry point invoked without an operator parameter, whose enforcement consults only `detectTransferRestriction(address(0), to, value)`. Predicting such an operation with `detectTransferRestrictionFrom` can surface an operator restriction that the entry point never applies, causing an integrator to withhold a transaction that would have succeeded. Integrators SHOULD select the predictor the implementation designates for the entry point they intend to call.
- An implementation that reuses one address-based policy for both transfers and supply-changing operations should be careful about how it makes the mint case pass. Satisfying a sender-side check by adding `address(0)` to an allow-list also makes `address(0)` an acceptable *recipient* under the same policy, so a transfer to `address(0)` is no longer restricted — an unintended burn path on any token that does not independently reject zero-address recipients. Policies SHOULD branch on the `address(0)` encoding explicitly instead, treating a mint as an operation with no sender rather than as a transfer from a holder that happens to be allow-listed.
- A restricted token that consults a restriction source inherits that source's evaluation cost on every transfer, and where the source composes several policies that cost is unbounded in the number of policies. An administrator adding policies without bound, or an adversary able to influence how many are evaluated, can push transfers past the block gas limit and halt the token — a denial of service on the transfer path itself, not merely on the predictor. Implementations SHOULD bound the cost of policy evaluation, and SHOULD treat unbounded iteration inside the transfer path as a griefing vector.
- Restriction codes are not stable identifiers across time. Reordering a policy set, replacing one policy with another, or upgrading a restriction source can change which code is returned for the same underlying cause, and the determinism required of restriction checks is defined for the same state and inputs — it does not constrain what happens across state changes. Integrators MUST NOT cache a code-to-meaning mapping across time and MUST re-read `messageForTransferRestriction` rather than relying on a previously observed message for the same code.
- A `detectTransferRestriction` that reverts rather than returning a code turns the reporting result into a third outcome that the enforcement-consistency requirement does not describe, and can break a caller that enumerates codes or performs a pre-flight check. This is a realistic failure mode for a token consulting an external restriction source: an unset, paused, or self-destructed source will revert rather than return a code. Integrators SHOULD fail closed and treat a reverting predictor as restricted, rather than as unrestricted or as an error to ignore.
- Returning machine-readable codes improves integration, but the string returned by `messageForTransferRestriction` remains informational and is not an authorization primitive.
- Using `uint8` for restriction codes limits the available code space to 256 values (`0` to `255`), which can create ambiguity if code allocation is not managed carefully.
- ERC-1404 interface support alone is not evidence that a contract implements a full token interface such as ERC-20 or ERC-777; a restriction source can expose the ERC-1404 restriction interface and interface support while implementing no token transfer behavior at all. Callers

MUST NOT assume that a contract reporting ERC-1404 support is itself a transferable token, and MUST NOT read a `0` from a restriction source as a prediction about any particular token's transfer path: the source answers only for its own policy, while the token that consults it may impose others.

- The original ERC-1404 text did not require ERC-165 signaling. Therefore, older implementations may still implement ERC-1404 while not returning `true` for the ERC-165 interface identifier `0xab84a5c8`.

Copyright

Copyright and related rights waived via CC0.